IntelliShell

AI-Powered Natural Language Terminal Shell

OPERATING SYSTEMS COURSE PROJECT REPORT

Generated: October 28, 2026

1. Executive Summary & Project Overview

IntelliShell is an intelligent, interactive command-line shell written in C++ that integrates Artificial Intelligence (Google Gemini) to bridge the gap between human natural language and low-level operating system commands. Instead of memorizing obscure terminal syntax, flags, or utility names, users express their intent in plain English (e.g., 'show all files in the current folder' or 'check network ip').

The project demonstrates a dual-process architecture where a high-performance compiled C++ shell manages the terminal interface, process lifecycle, and kernel execution, while a decoupled Python adapter interfaces with cloud LLM endpoints and structured JSON validation.

Core Architectural Objectives:

- **Interactive C++ Terminal Interface:** Renders dynamic colored prompts ([user@host] - [cwd]), handles user input, command history, and custom execution loops.
- **AI Natural Language Translation:** Converts English instructions into precise operating system commands (PowerShell/CMD on Windows, POSIX /bin/sh on Linux).
- **Human-in-the-Loop Safeguard:** Displays the AI-generated command alongside an explanation. Users can Run (r), Edit (e), or Cancel (c) before execution.
- **Proactive Security Filter:** Scans commands for destructive patterns (e.g., fork bombs, recursive deletes, partition wipes) and enforces explicit confirmation.
- **Resilient Offline Fallback:** Includes local keyword/regex translation so that network timeouts or missing API keys never crash the shell.

2. Operating System Concepts Applied in IntelliShell

This project implements core Operating System principles studied in computer science courses. Below is a direct mapping of each theoretical concept to its concrete implementation in the codebase:

2.1 Process Creation and Lifecycle Management

In modern operating systems, processes run in isolated address spaces. The C++ shell creates and manages child processes dynamically using native kernel APIs:

- **Windows (CreateProcessA):** Spawns python.exe as a child process with custom STARTUPINFO.
- **Linux (fork + exec):** Uses fork() to duplicate the parent process and execlp() to overwrite the child's address space with the Python interpreter image.

```
// Windows Process Creation in ai_shell.cpp
CreateProcessA(NULL, cmdVec.data(), NULL, NULL, TRUE, 0, NULL, NULL, &siStartInfo, &piProcInfo);
```

2.2 Inter-Process Communication (IPC) via Anonymous Pipes

Because the C++ shell and the Python AI adapter are distinct processes with independent memory spaces, they communicate across kernel-managed anonymous pipes (unnamed byte streams):

- **Anonymous Pipes:** C++ creates dual unidirectional pipes (one for STDIN and one for STDOUT). The parent writes JSON requests to the child's STDIN pipe and reads structured JSON responses from the child's STDOUT pipe.

```
// Creating kernel pipe buffers in ai_shell.cpp
CreatePipe(&hChildStd_OUT_Rd, &hChildStd_OUT_Wr, &saAttr, 0);
CreatePipe(&hChildStd_IN_Rd,  &hChildStd_IN_Wr,  &saAttr, 0);
```

2.3 Standard I/O Redirection (File Descriptors & Handles)

Normally, standard streams point to the console keyboard and display. The shell programmatically redirects standard handles prior to child execution:

- **Handle Remapping:** Configures STARTUPINFO with STARTF_USESTDHANDLES, routing Python's stdin/stdout to the pipe endpoints rather than the terminal.

```
siStartInfo.hStdInput  = hChildStd_IN_Rd;   // Redirect STDIN
siStartInfo.hStdOutput = hChildStd_OUT_Wr;  // Redirect STDOUT
siStartInfo.dwFlags   |= STARTF_USESTDHANDLES;
```

2.4 Process Synchronization & Zombie Prevention

A well-behaved shell must synchronize with child processes to prevent race conditions and zombie entries in the operating system's process table:

- **Blocking Wait:** WaitForSingleObject (Windows) or waitpid (Linux) halts parent execution until the child signals termination, after which handles are safely closed.

```
WaitForSingleObject(piProcInfo.hProcess, INFINITE);
CloseHandle(piProcInfo.hProcess);
CloseHandle(piProcInfo.hThread);
```

2.5 Working Directory & Environment Block System Calls

The shell interacts directly with the Process Environment Block (PEB) and OS directory tables:

- **System Calls:** Queries getenv() for USER/HOME context, calls _getcwd() to locate the current directory, and executes _chdir() system calls when handling directory navigation.

2.6 OS Security & Protection Guard

To protect host system integrity, IntelliShell acts as a security filter before issuing commands to the OS:

- **Safety Interception:** Matches regular expressions against dangerous signatures (fork bombs ':(){ :|:& };:', recursive wipe 'Remove-Item -Recurse', volume formatting) to enforce explicit user confirmation.

3. Step-by-Step Execution Demo

The following sequence demonstrates how a user instruction is processed through the entire pipeline:

Live Execution Trace: 'show all files in the current folder'

```
[Riya gupta@LAPTOP-Q5R8L2TO] - [~\Downloads\OS CP 2\IntelliShell]
> show all files in the current folder

[AI Suggestion] [Command]
$ Get-ChildItem

[Explanation]
Lists all files and folders in the current directory on Windows (PowerShell/CMD).

[Action] Run as-is (r), edit (e), or cancel (c)? r

[Output]
    Directory: C:\Users\Riya gupta\Downloads\OS CP 2\IntelliShell
Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        27-10-2026  10.49 PM                nlohmann
-a----        27-10-2026  11.34 PM             69 .env
-a----        27-10-2026  11.50 PM           4457 ai_adapter.py
-a----        27-10-2026  11.26 PM          12240 ai_shell.cpp
-a----        27-10-2026  11.27 PM        2304195 ai_shell.exe
-a----        27-10-2026  11.30 PM            301 run.bat
```

4. Future Scope & Advanced OS Extensions (Threads & Concurrency)

To further elevate the project's academic and practical standing, several advanced Operating Systems concepts can be integrated into future versions:

4.1 Multithreading and Asynchronous AI Worker Threads (Priority)

Currently, when the shell queries the AI adapter, the main process waits synchronously. Introducing native C++ multithreading (std::thread, std::mutex, condition variables) provides enormous architectural benefits:

- **Non-Blocking UI Thread:** The main thread remains fully responsive, handling user keystrokes, window resizing, or immediate command cancellation.
- **Asynchronous Worker Thread:** A dedicated background thread manages pipe I/O and network latency with the AI adapter, signaling the main thread via thread-safe queues.
- **Real-Time Visual Feedback:** Enables the main thread to render an animated console progress indicator (spinner) while waiting for the cloud AI response.

```
// Conceptual Multithreaded Architecture
std::mutex mtx;
std::condition_variable cv;
bool ai_ready = false;
std::string ai_response;

void worker_thread(std::string prompt) {
    std::string result = run_adapter("./ai_adapter.py", prompt);
    std::lock_guard<std::mutex> lock(mtx);
    ai_response = result;
    ai_ready = true;
    cv.notify_one();
}
```

4.2 Native Job Control & Background Execution ('&', jobs, fg, bg)

Classic shells support asynchronous process execution via the '&' operator. Future iterations will implement:

- **Job Table Management:** Tracking process groups, PIDs, and process states (Running, Stopped, Done).
- **Built-in Job Commands:** Adding 'jobs' to inspect background tasks and 'fg <PID>' / 'bg <PID>' to transfer processes between foreground and background terminal control.

4.3 Asynchronous Signal Handling (SIGINT, Ctrl+C)

Intercepting hardware interrupts via OS signal handlers (SetConsoleCtrlHandler on Windows / sigaction on Linux) ensures that typing Ctrl+C cleanly terminates a runaway child process without killing the shell itself.

4.4 Process Resource Accounting & Profiling

Querying kernel accounting metrics (GetProcessTimes / getrusage) after each command to report exact User-mode CPU time, Kernel-mode CPU time, and peak memory working set.

Conclusion:
IntelliShell effectively combines systems programming, OS system calls, IPC, and artificial intelligence to deliver a modern, secure, and user-friendly command-line experience.